

[Home](#)

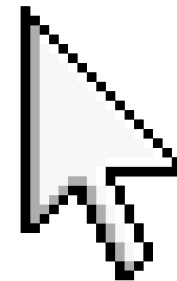
[Content](#)

[Conclusion](#)

CVIA'25: Course Project

SPACECRAFT DETECTION & SEGMENTATION

OF SPARK 2024 DATA



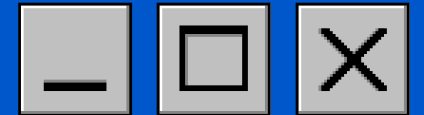
By:

Eyasu Araya
Isaías Antúnez García
Praveen Agnihotry
Maksym Voľoshyn

Start

Mentored By:
Hazem Ali





[Home](#)

[Content](#)

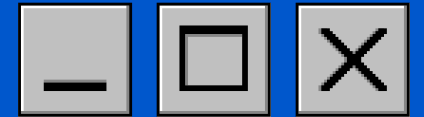
[Conclusion](#)



MASK R-CNN FOR SPACECRAFT DETECTION

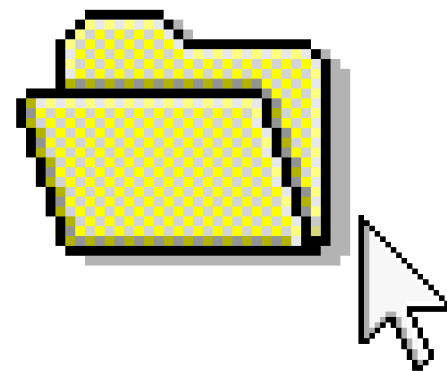


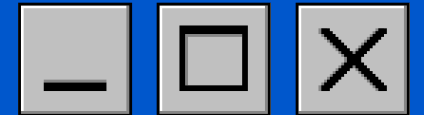
Mask R-CNN for Spacecraft Detection



Home **Content** Conclusion

- **Architecture:** ResNet-50 + FPN backbone
- **Key Insight:** Mask predictions → tighter bounding boxes
- **Final Score:** 0.9452 (ensemble + TTA)



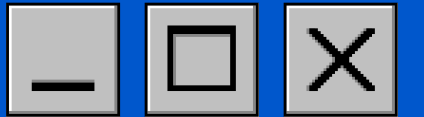


Approach	Score
Baseline	0.9379
+ Mask-guided boxes	0.9417
+ Ensemble + TTA	0.9452

Key Takeaways:

- **Simple training** > complex modifications
- **Inference-time optimization** gave biggest gain
- **Ensemble** provides consistent improvement





[Home](#)

[Content](#)

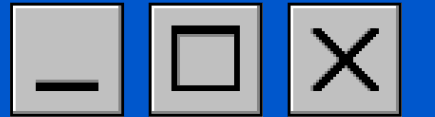
[Conclusion](#)



MASK R-CNN FOR SPACECRAFT SEGMENTATION



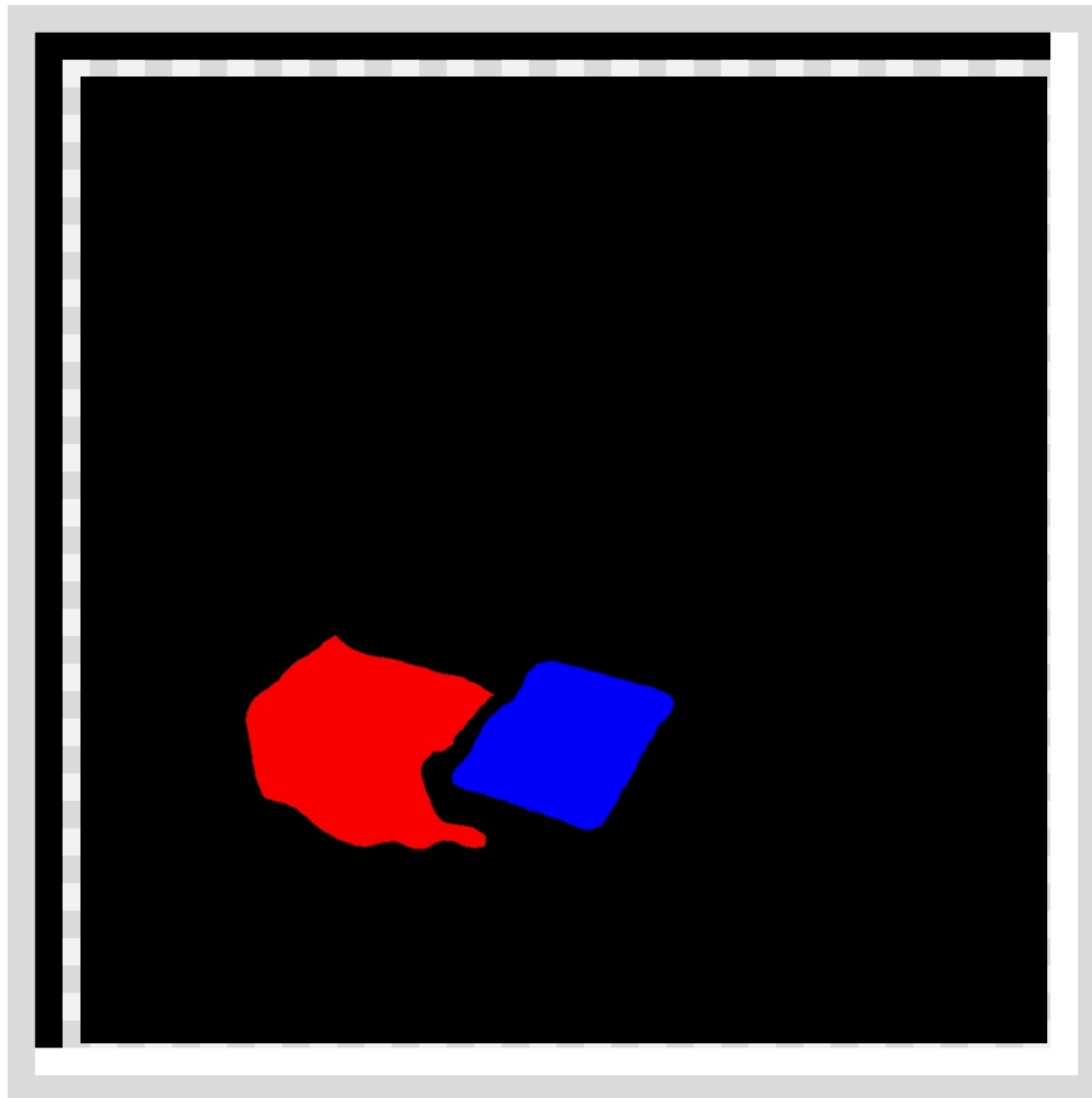
Mask R-CNN for Spacecraft Segmentation



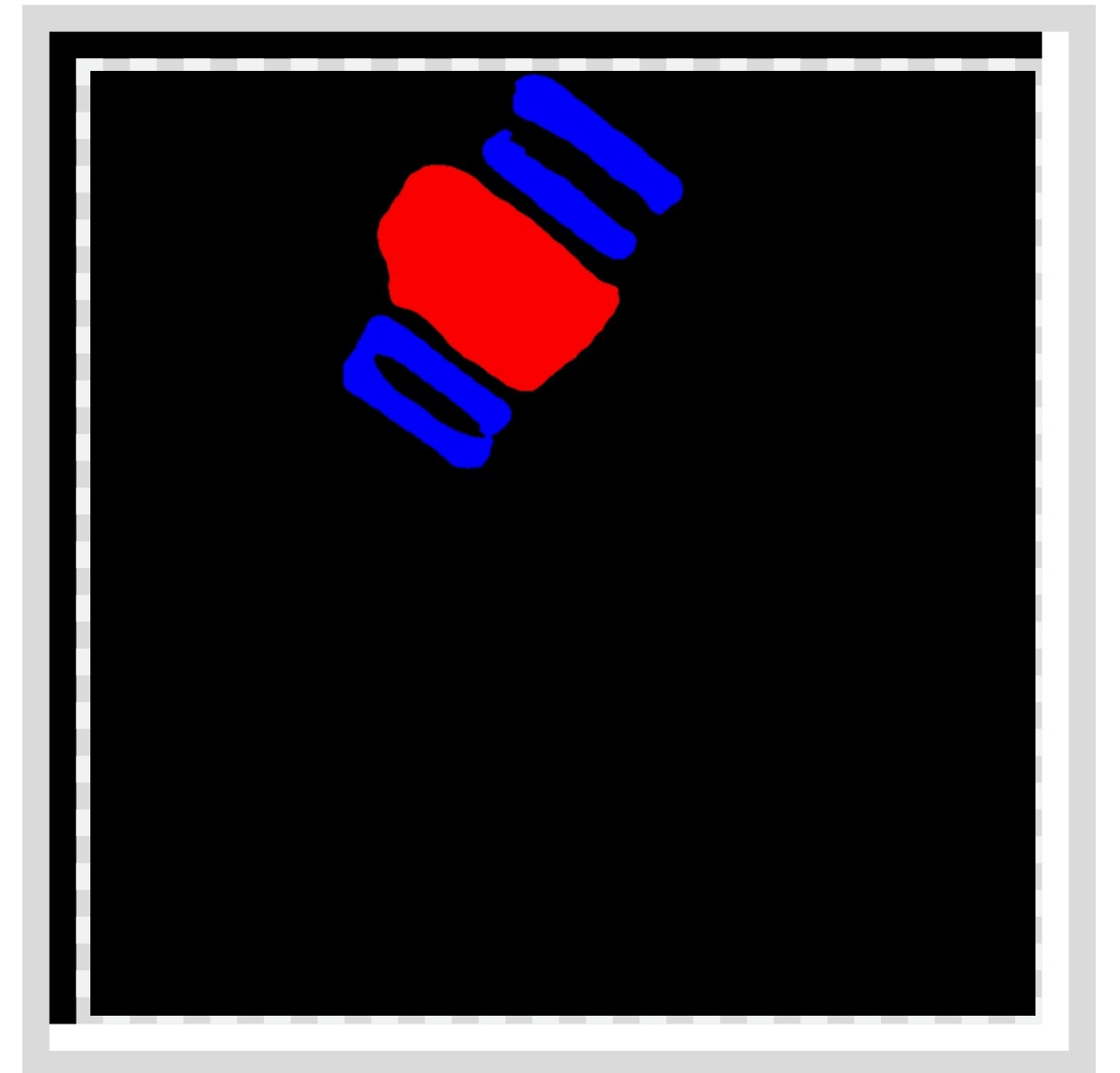
[Home](#)

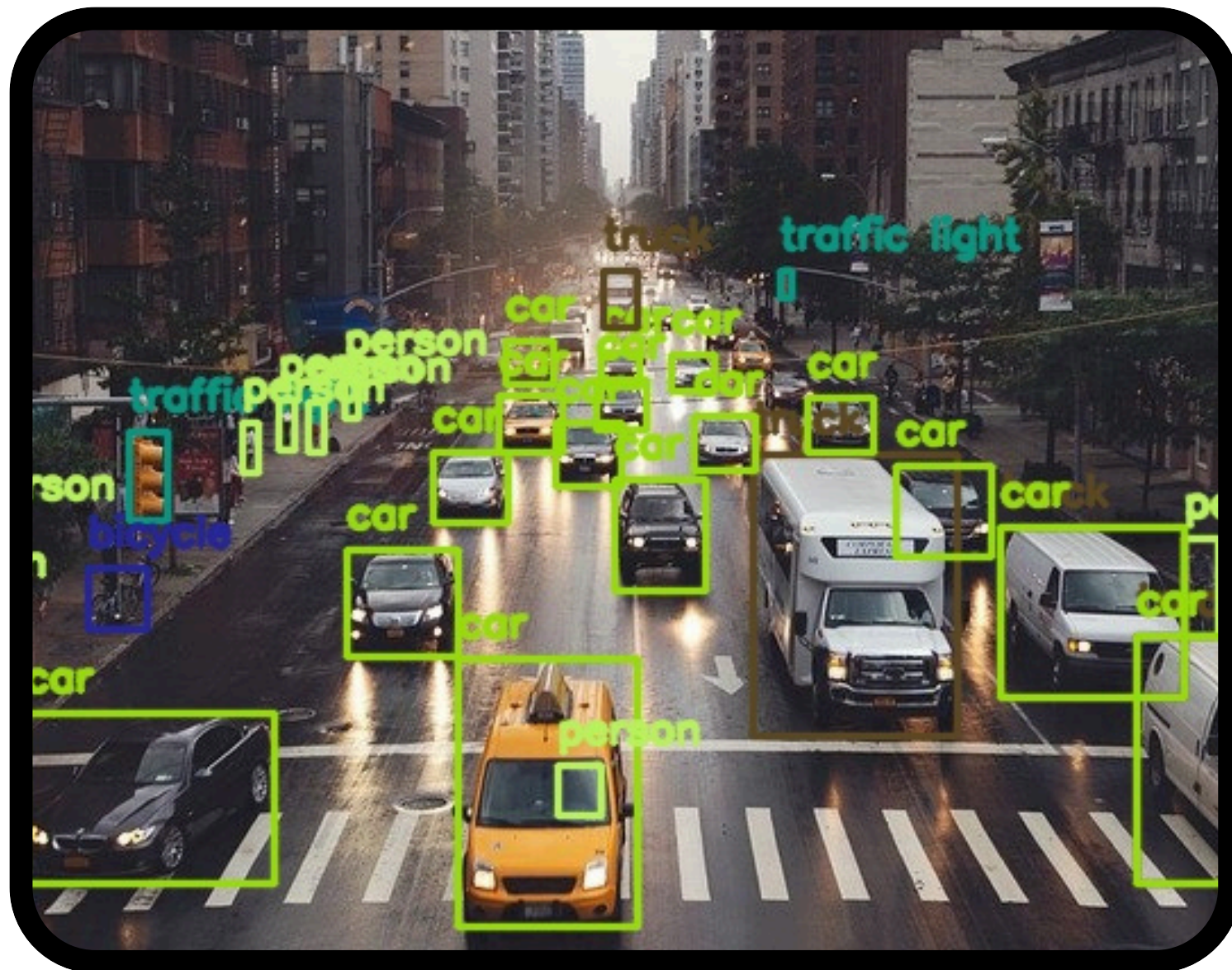
[Content](#)

[Conclusion](#)



- **Task:** Binary segmentation (Body vs Solar Panels)
- **Architecture:** Same Mask R-CNN (ResNet-50 + FPN)
- **Output:** Color-coded masks (Red = Body, Blue = Panels)
- **Score:** 0.64





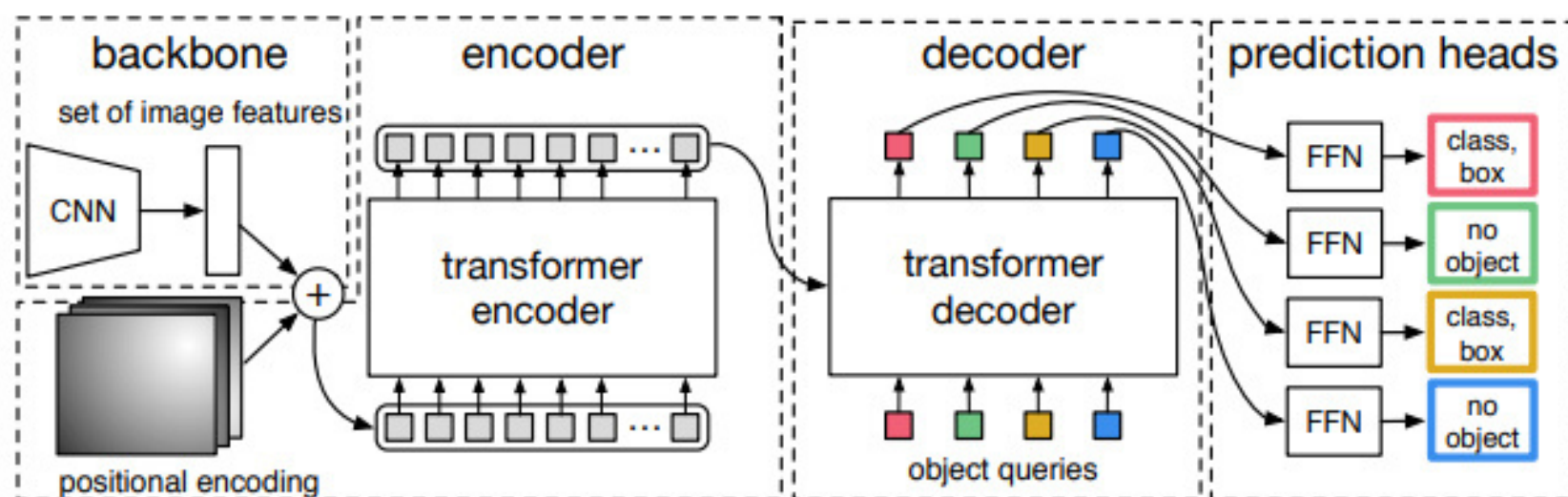
WHY NOT RT-DETR?



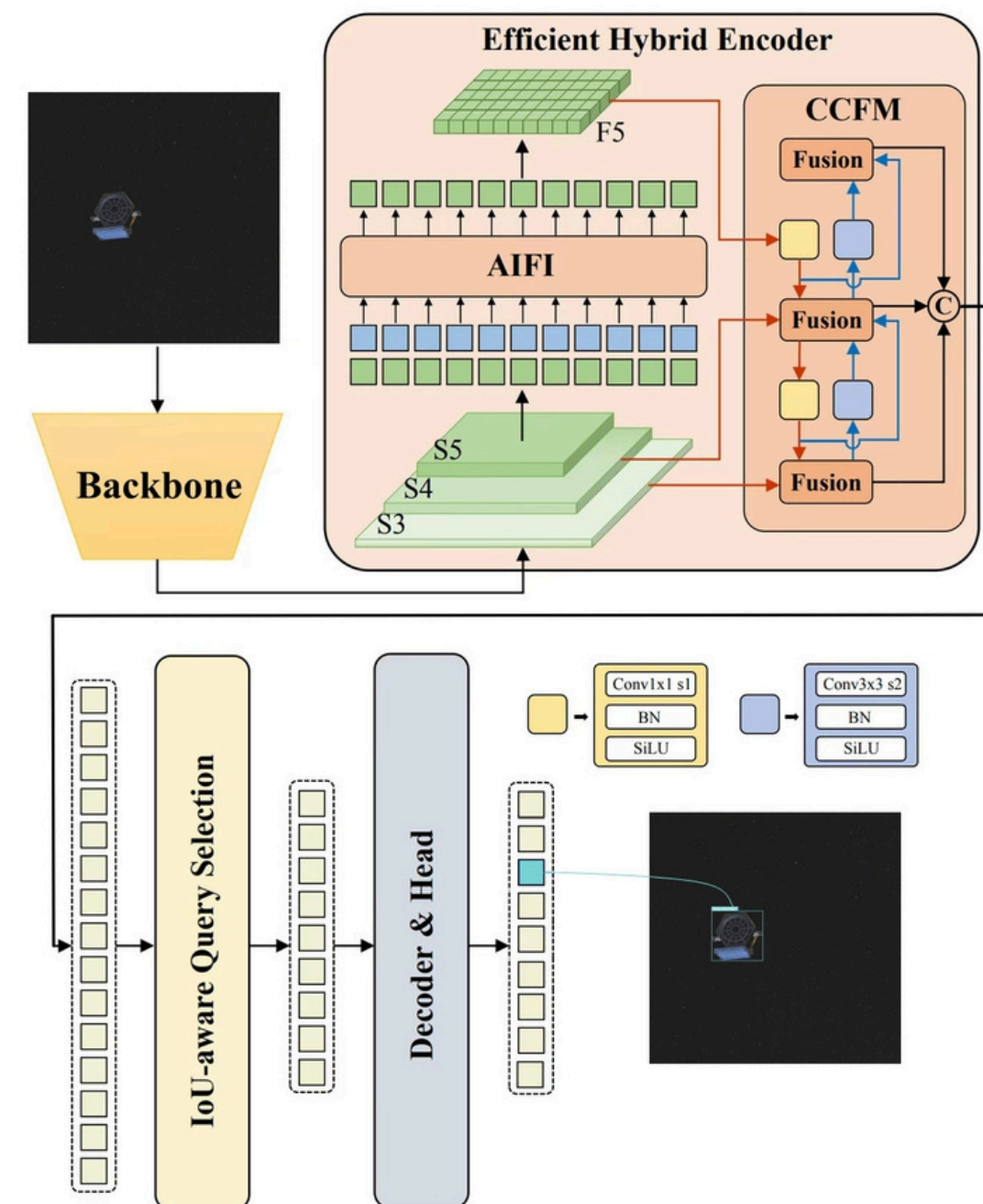
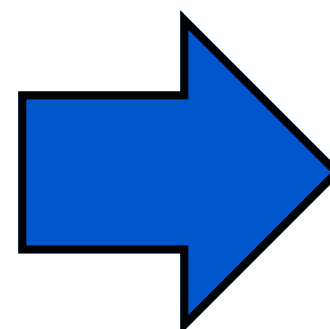
**REAL TIME
DETECTION
TRANSFORMER**



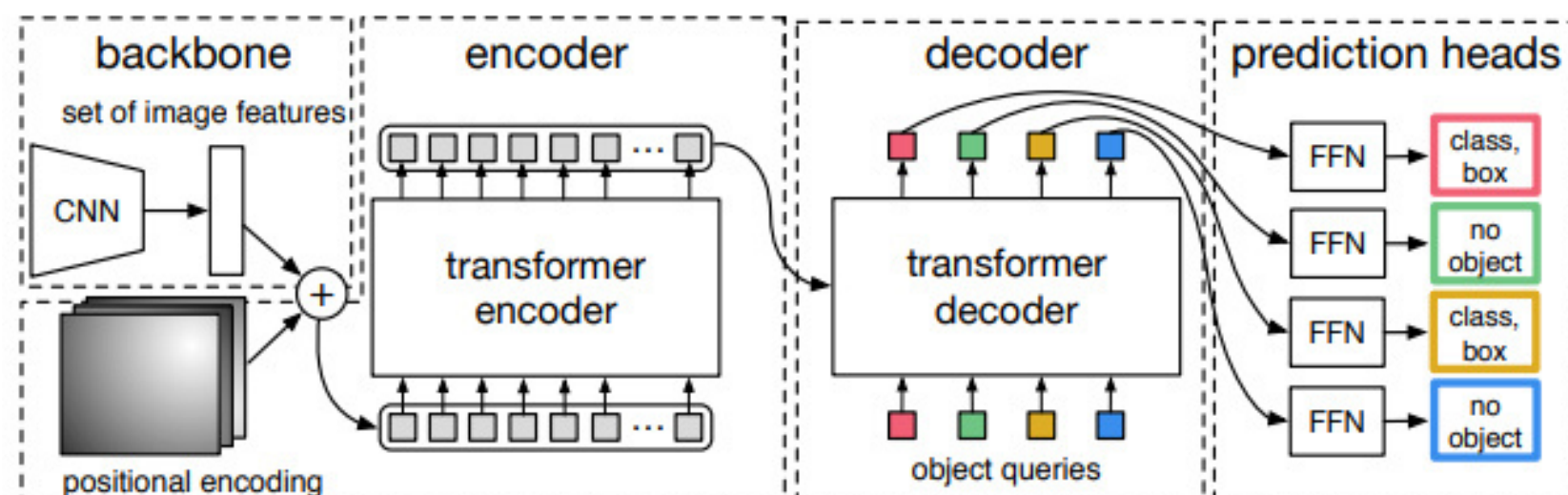
Original DETR



RT-DETR

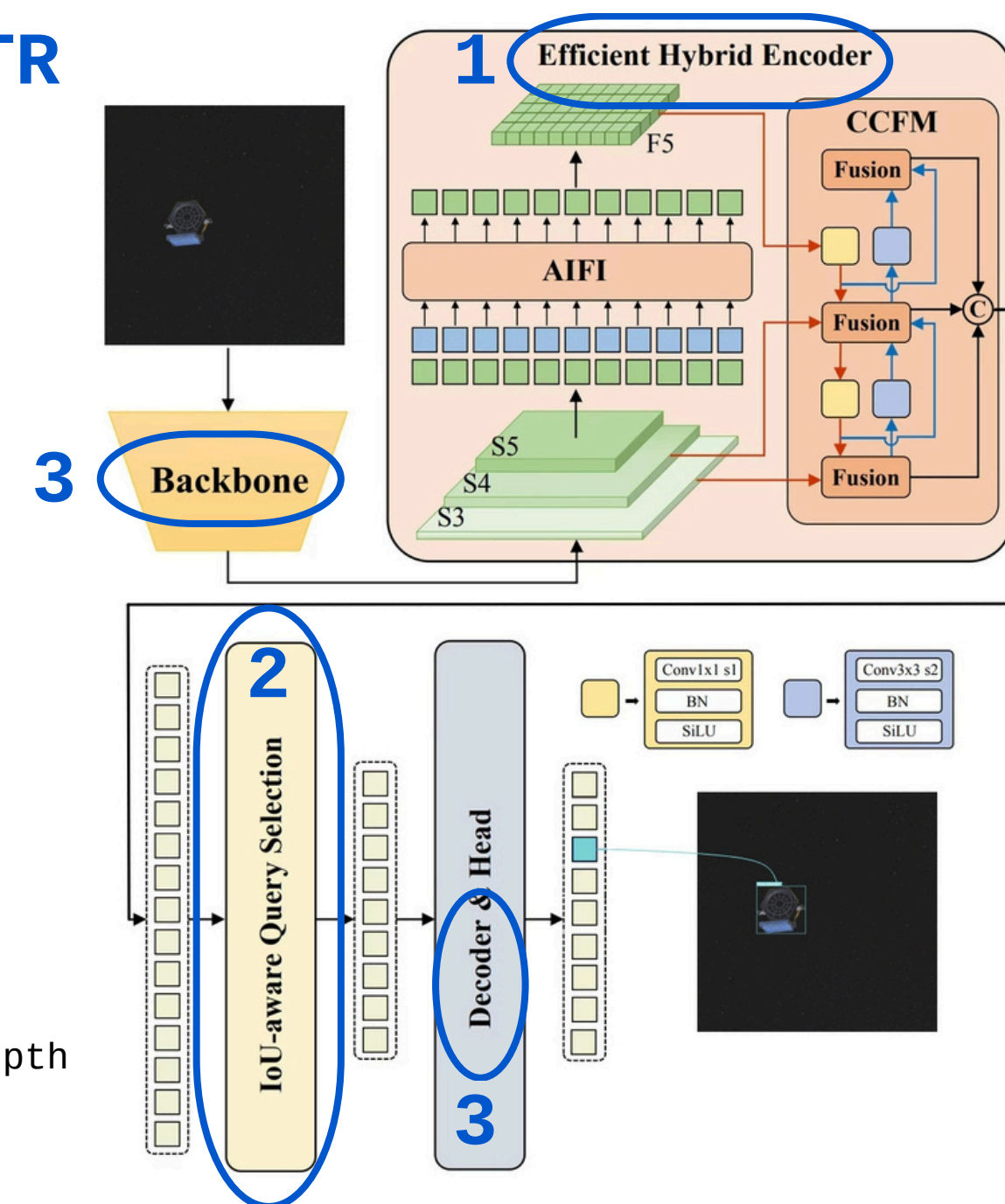


Original DETR



1. **Efficient Hybrid Encoder:** decouples intra-scale interaction/cross-scale fusion
2. **IoU-Aware Query Selection:** high-quality initial queries to the decoder
3. **Scalable Architecture:** interchangeable backbones (ResNet/HGNetv2) & adjustable depth

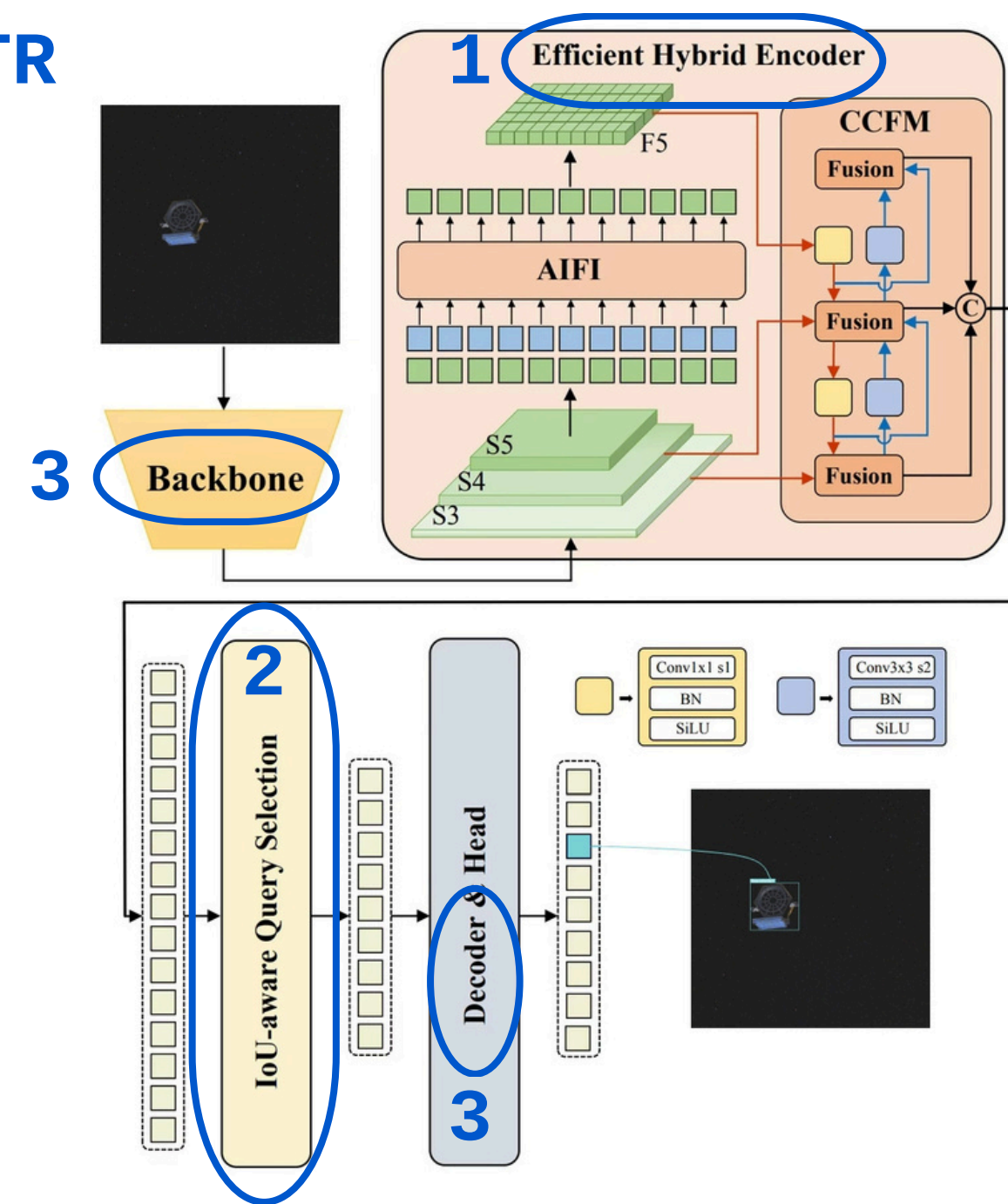
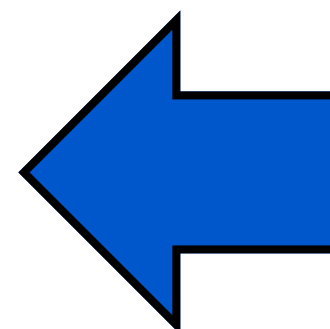
RT-DETR

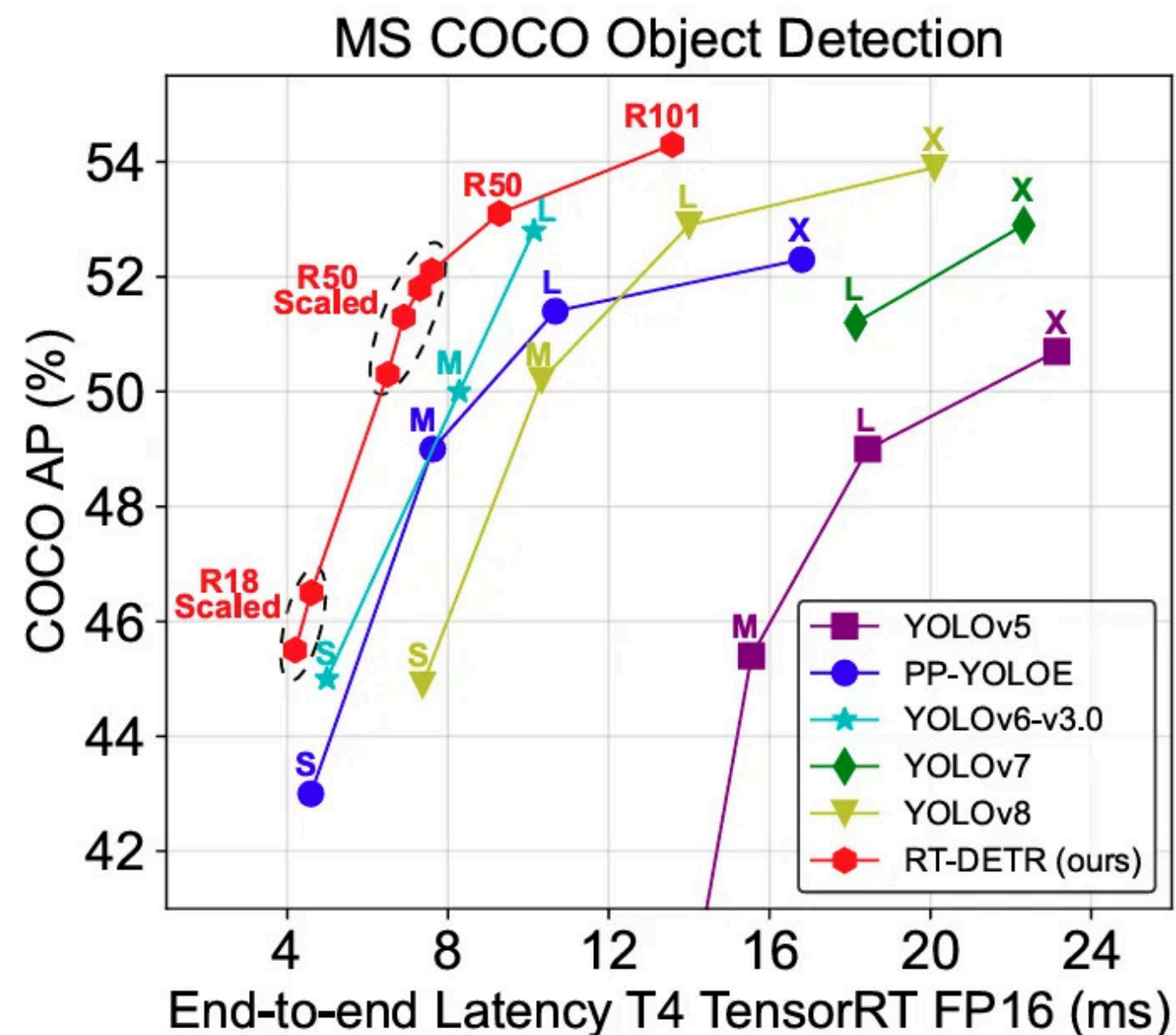


BENEFITS

1. Faster Convergence
2. Real-Time Inference
3. NMS-Free Detection
4. Flexible Speed-Accuracy Trade-off

RT-DETR



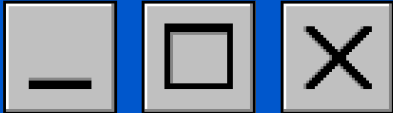


RT-DETR achieves superior accuracy (54% COCO AP) with competitive latency (6-10ms), beating YOLO family

- **Best accuracy:** 54% COCO AP
- **Fastest inference:** ~4ms latency
- **Sweet spot:** RT-DETR R50 Scaled: 52% AP with only 6ms latency

[Source](https://arxiv.org/abs/2304.08069): Zhao et al. DETRs Beat YOLOs on Real-time Object Detection. [arXiv:2304.08069](https://arxiv.org/abs/2304.08069), 2024.





Hyperparameters:

- **Optimizer:** AdamW with learning rate $lr = 1 \times 10^{-5}$
- **Learning rate schedule:** Cosine annealing with $T_{max} = 35$, $\eta_{min} = 1 \times 10^{-6}$
- **Warmup:** Linear warmup for 2000 iterations
- **Batch size:** 5 (limited by RTX 3080 Mobile 8GB VRAM)
- **Training epochs:** 35 total
- **Loss weights:** Varifocal Loss (VFL) = 1.0, L1 bbox = 5.0, GIoU = 2.0
- **EMA:** Exponential Moving Average with decay = 0.9999, warmup = 2000 steps
- **Weight decay:** 1×10^{-4}

Data Augmentation Pipeline:

- **RandomPhotometricDistort** ($p = 0.5$): Random brightness, contrast, saturation, and hue adjustments
- **RandomZoomOut**: Random zoom-out with background padding to simulate varying distances
- **RandomIoUCrop** ($p = 0.8$): IoU-aware random cropping preserving object visibility
- **RandomHorizontalFlip** ($p = 0.5$): Horizontal mirroring for orientation invariance
- **SanitizeBoundingBoxes**: Post-augmentation bbox validation with minimum size threshold
- **Multi-Scale Training** - seven resolutions: {640, 672, 704, 736, 768, 800, 832}

TABLE III
SPARK 2024 TRAINING DATASET STATISTICS

Property	Training Set	Validation Set
Total images	60,000	20,000
Image resolution	1024 × 1024	1024 × 1024
Classes	10	10
Instances per class	6,000	2,000
Objects per image	1.0	1.0

TABLE IV
RT-DETR CHECKPOINT PERFORMANCE

Checkpoint	Epoch	Validation Score	Description
Early convergence	14	0.9117	Rapid learning phase
Best model	27	0.9674	Peak performance
Final checkpoint	35	0.9672	Post-plateau



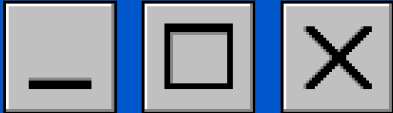


TABLE V
DETECTION METHOD COMPARISON

Method	Backbone	Epochs	Score	Key Advantage
Mask R-CNN	ResNet-50+FPN	48	0.9452	Robust local.
DETR	ResNet-101	150	0.6392	Global context
RT-DETR	ResNet-50-vd	35	0.9674	Efficiency+Acc.

RT-DETR significantly outperformed both CNN and transformer baselines.



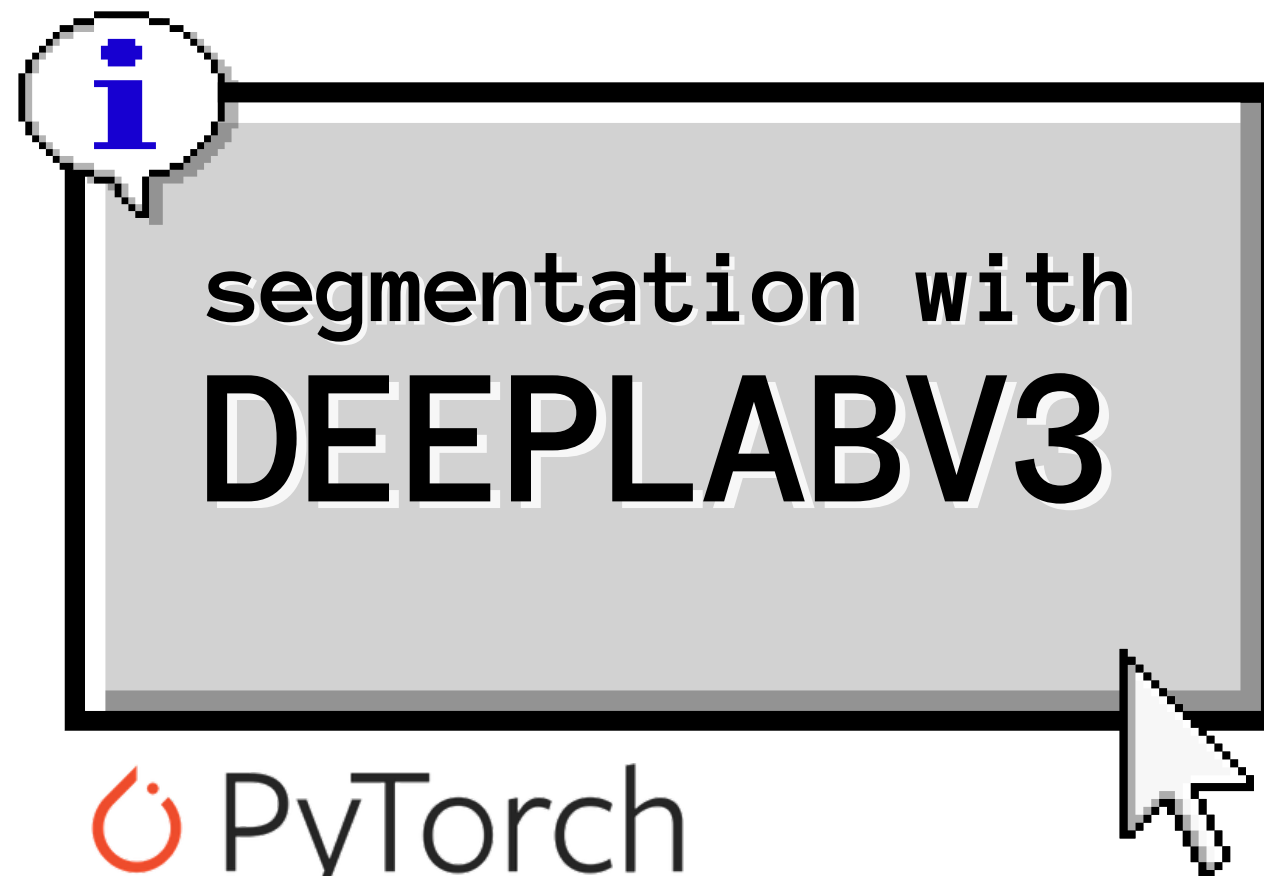
Performance Advantages:

- **+2.4%** absolute improvement over **Mask R-CNN** through superior global reasoning for fine-grained spacecraft classification
- **+51.3%** absolute improvement over **DETR** through efficient hybrid encoder and IoU-aware queries
- **2.8× faster inference** than Mask R-CNN (42 FPS vs 15 FPS)
- **27% fewer epochs** than Mask R-CNN (35 vs 48) for superior performance

Success Factors:

- **Efficient Architecture:** Hybrid encoder enabled real-time performance without accuracy sacrifice
- **Rapid Convergence:** Achieved peak performance in 27 epochs vs DETR’s 500+ requirement
- **Transfer Learning:** COCO-pretrained weights provided robust initialization
- **Multi-Scale Robustness:** Seven-scale training strategy improved generalization
- **Augmentation Scheduling:** Aggressive augmentation(epochs 1-23) + clean fine-tuning (epochs 24-35) prevented overfitting





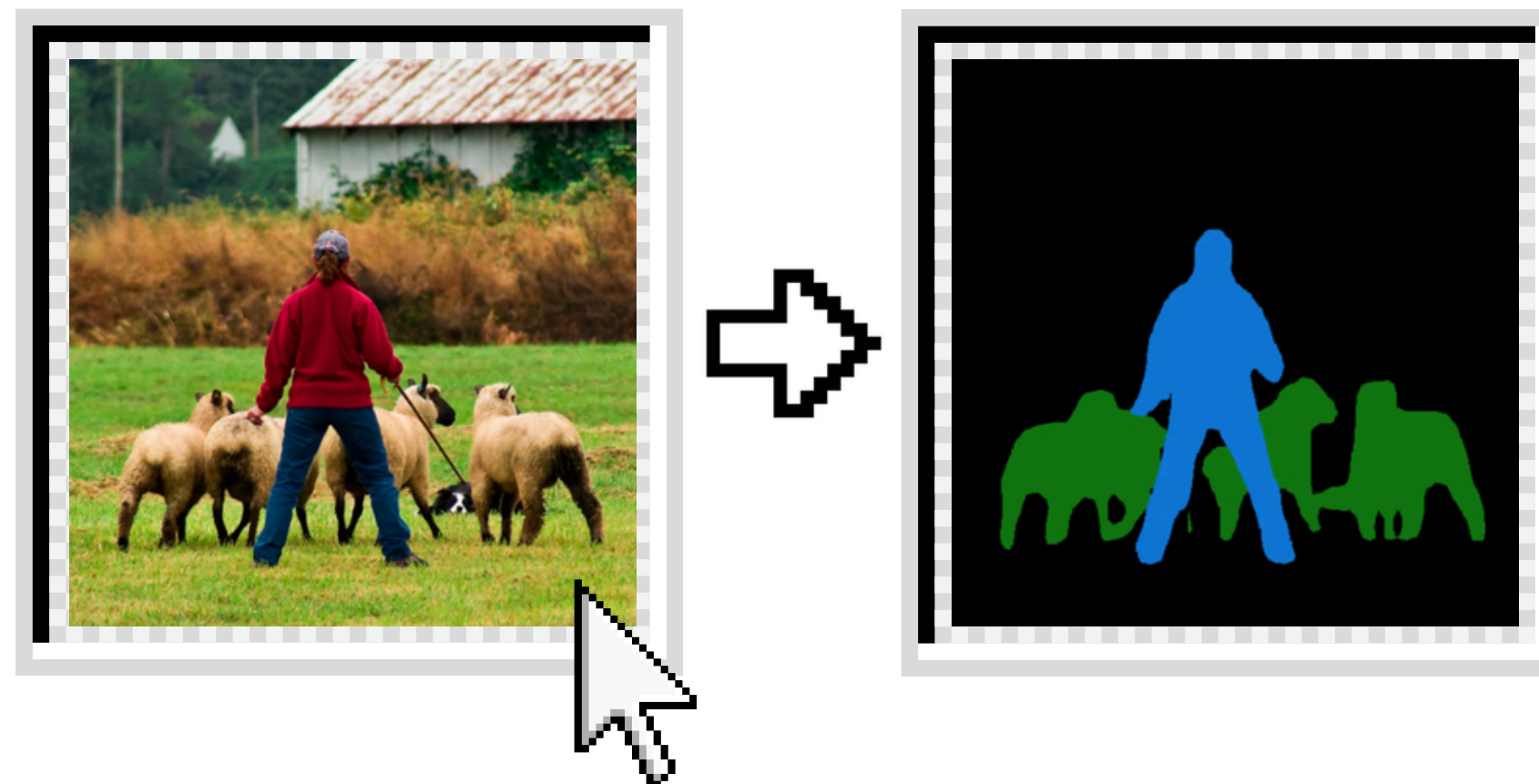
 PyTorch


DeepLabv3 is a Deep Neural Network (DNN) architecture for Semantic Segmentation Tasks. It uses Atrous (Dilated) Convolutions to control the receptive field and feature map resolutions without increasing the total number of parameters.



IMPLEMENTATION DETAILS OF THE MODEL

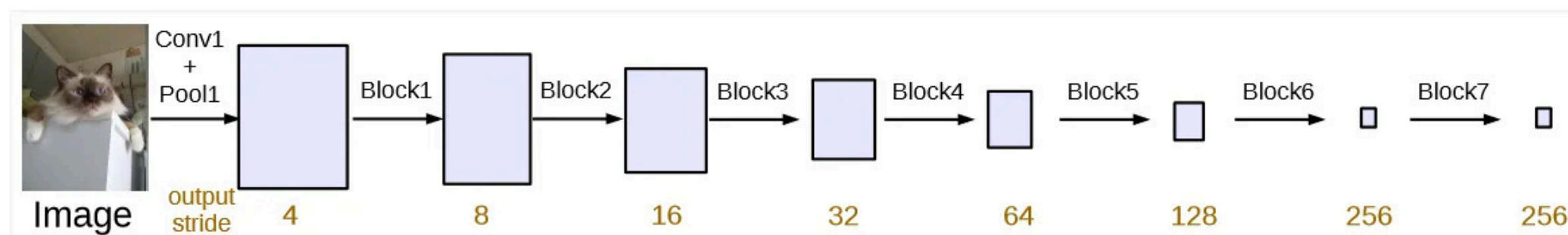
Deeplabv3-ResNet is constructed by a Deeplabv3 model using a ResNet-50 backbone. It is pretrained on the COCO dataset.



```
deeplabv3_resnet50(*[, weights, progress, ...])
```

Constructs a DeepLabV3 model with a ResNet-50 backbone.





(a) Going deeper without atrous convolution.

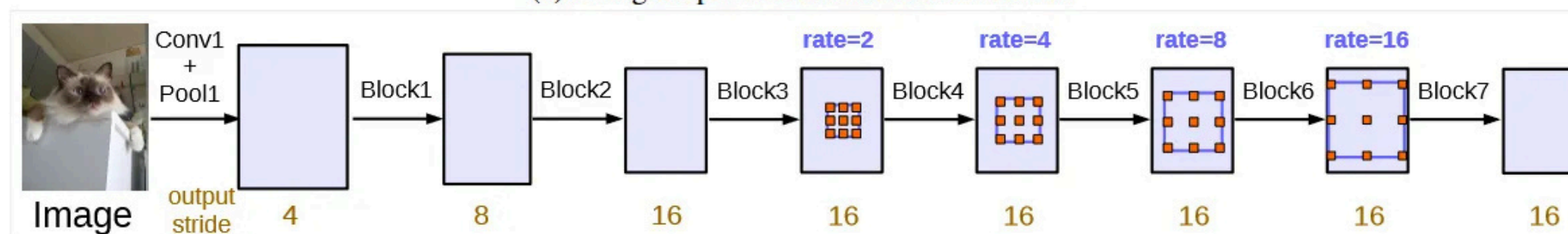
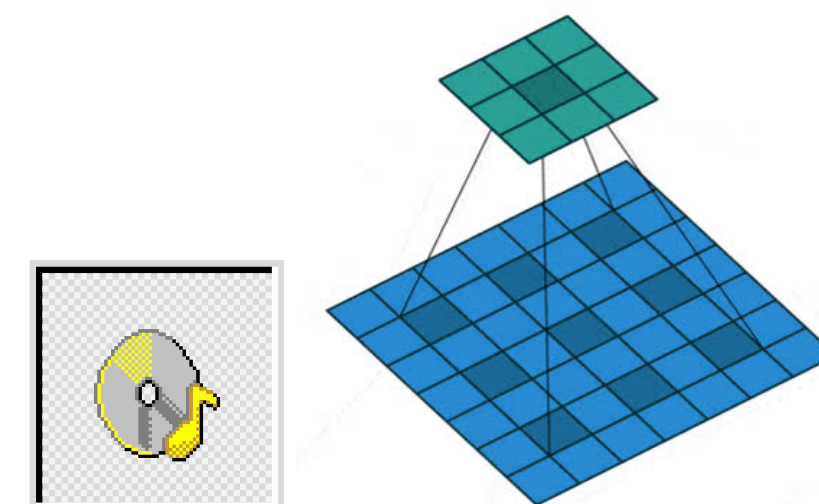
(b) Going deeper with atrous convolution. Atrous convolution with $rate > 1$ is applied after block3 when $output_stride = 16$.

Figure 3. Cascaded modules without and with atrous convolution.

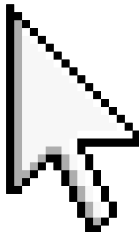
Atrous Convolution helps us construct a deeper network that **retains** more **high level information** at finer resolutions **without increasing** the number of **parameters**. A network with a higher output stride will be able to extract better and higher resolution features.

In atrous convolution, we just insert zeros into the convolution kernel to **increase** the size of the **kernel without increasing** the number of learnable **parameters** (because we don't care about the zeros).

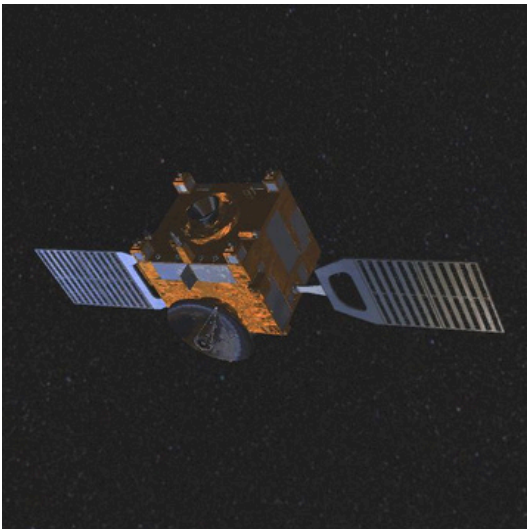
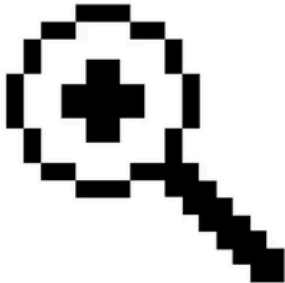




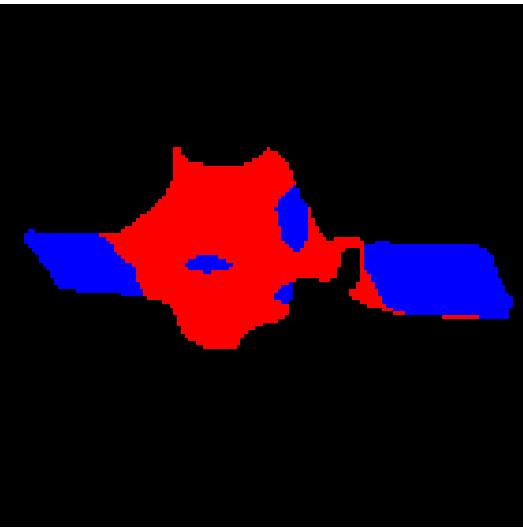
SEGMENTATION RESULTS



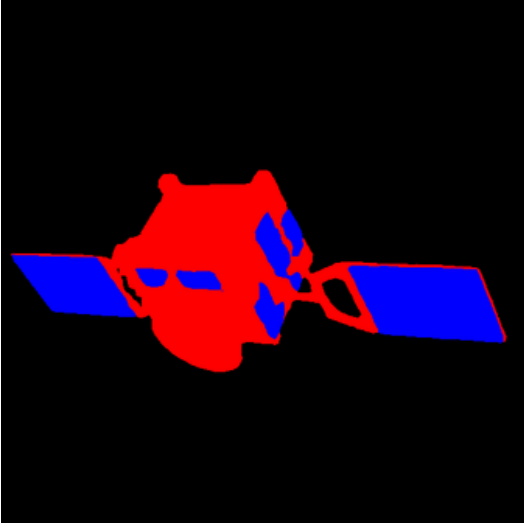
Input resolution	Epochs	Score	Notes
512 × 512	10	0.84	Resized training and inference.
1024 × 1024	10	0.88	Full resolution, shorter training.
1024 × 1024	20	0.91	Extended training at full resolution.



Test input



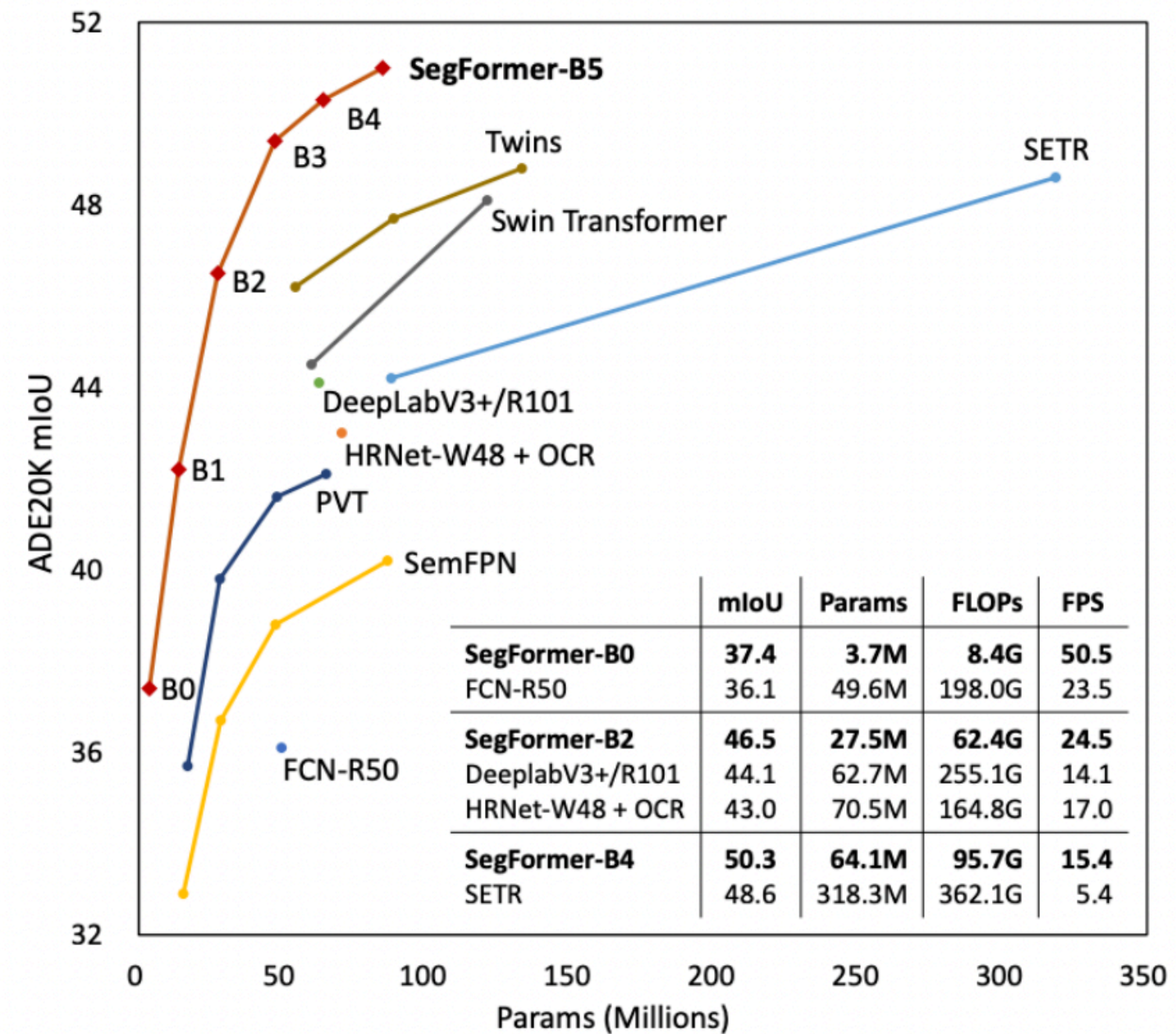
Upscaled 512 px



Full resolution



SegFormer is a semantic segmentation architecture designed to be simple, efficient, and accurate.



Performance vs. Efficiency on ADE20K (Credit: [Xie](#))

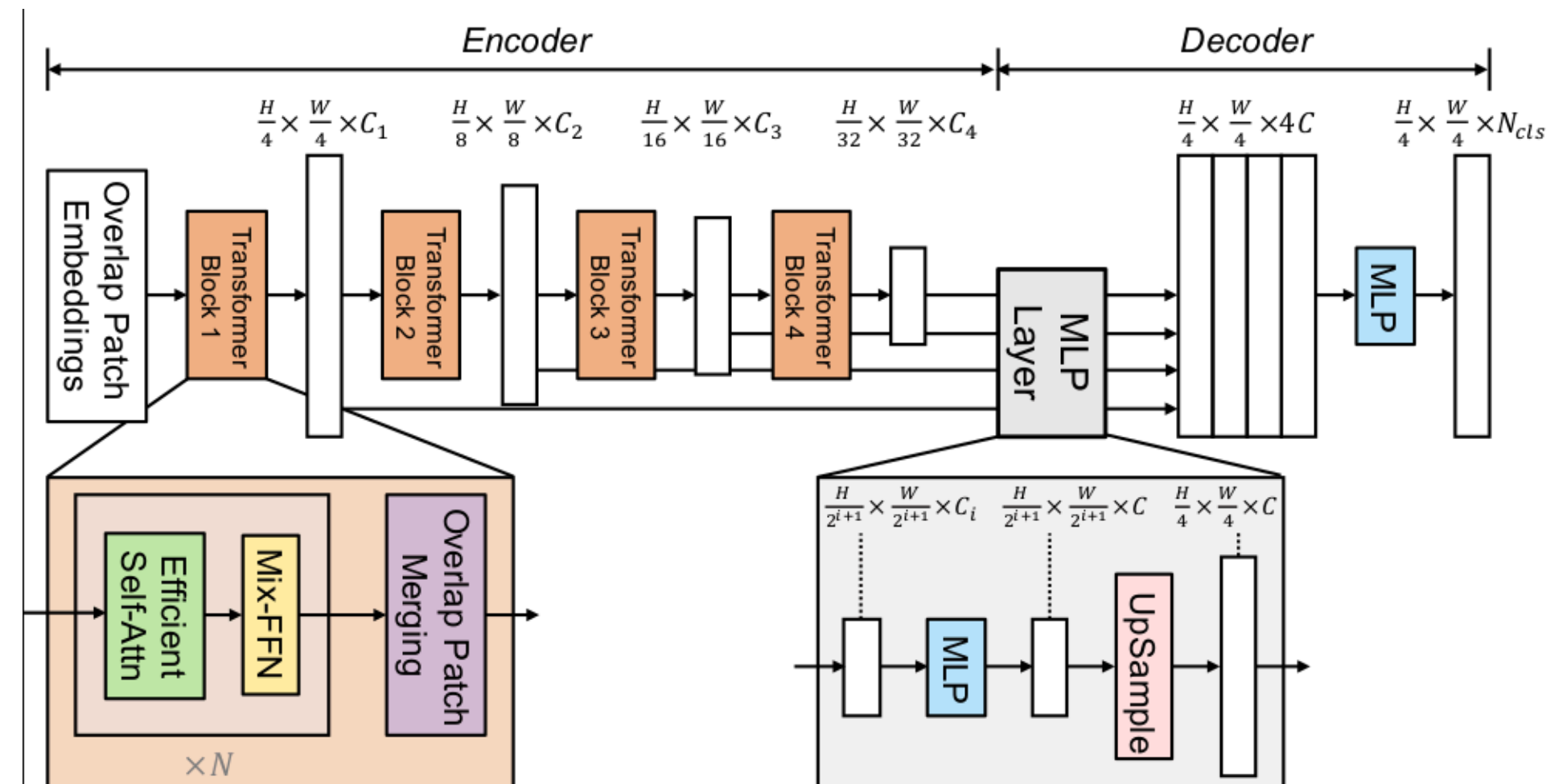


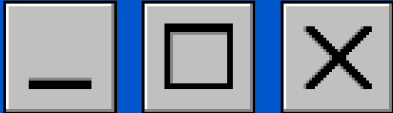
SegFormer has two main parts:

1. Encoder for feature extraction
2. Decoder converts features into pixel-level class predictions.

Why is Segformer special:

1. A hierarchical Transformer encoder (MiT)
2. A simple MLP-based decoder.
3. No positional encoding
4. No upsampling-heavy decoder





Training Configuration:

Category	Setting
Framework	PyTorch Lightning with DDP
Hardware	4 GPUs
Epochs	30
Early Stopping	Patience = 5
Optimizer	Adam
Learning Rate	6×10^{-5}
Loss Function	Standard Cross-Entropy(For baseline)
Checkpointing Metric	Validation mIoU
Image Resizing	None
Models Tested	SegFormer-B2, SegFormer-B4

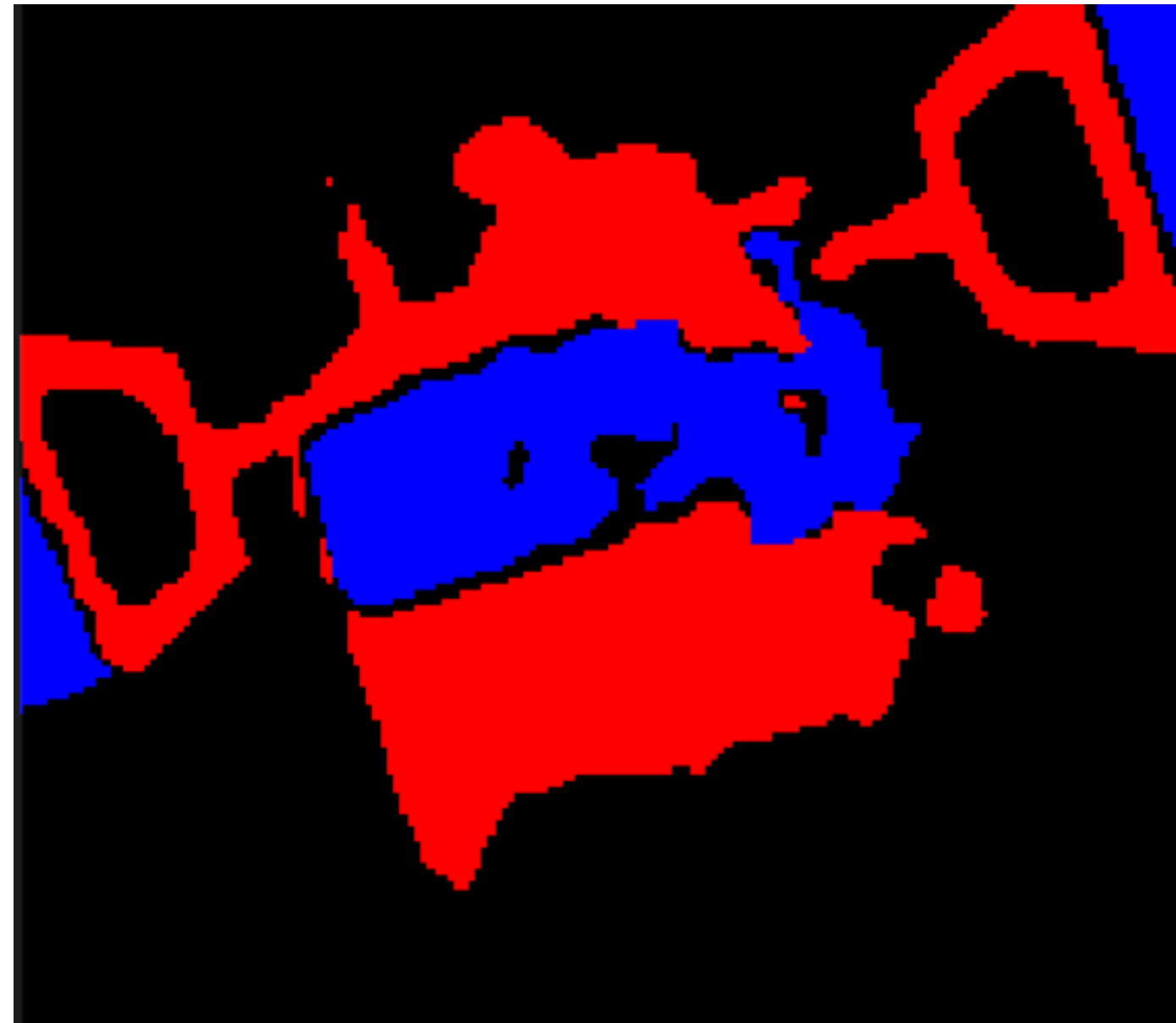


Baseline Mask Labeling:

- RGB-based thresholding
- Body: Red channel > 180 , Green & Blue < 50
- Solar panels: Blue channel > 180 , Red & Green < 50
- Background otherwise.

Results:

- Codabench score: 89%
- bias toward background



Class Imbalance:

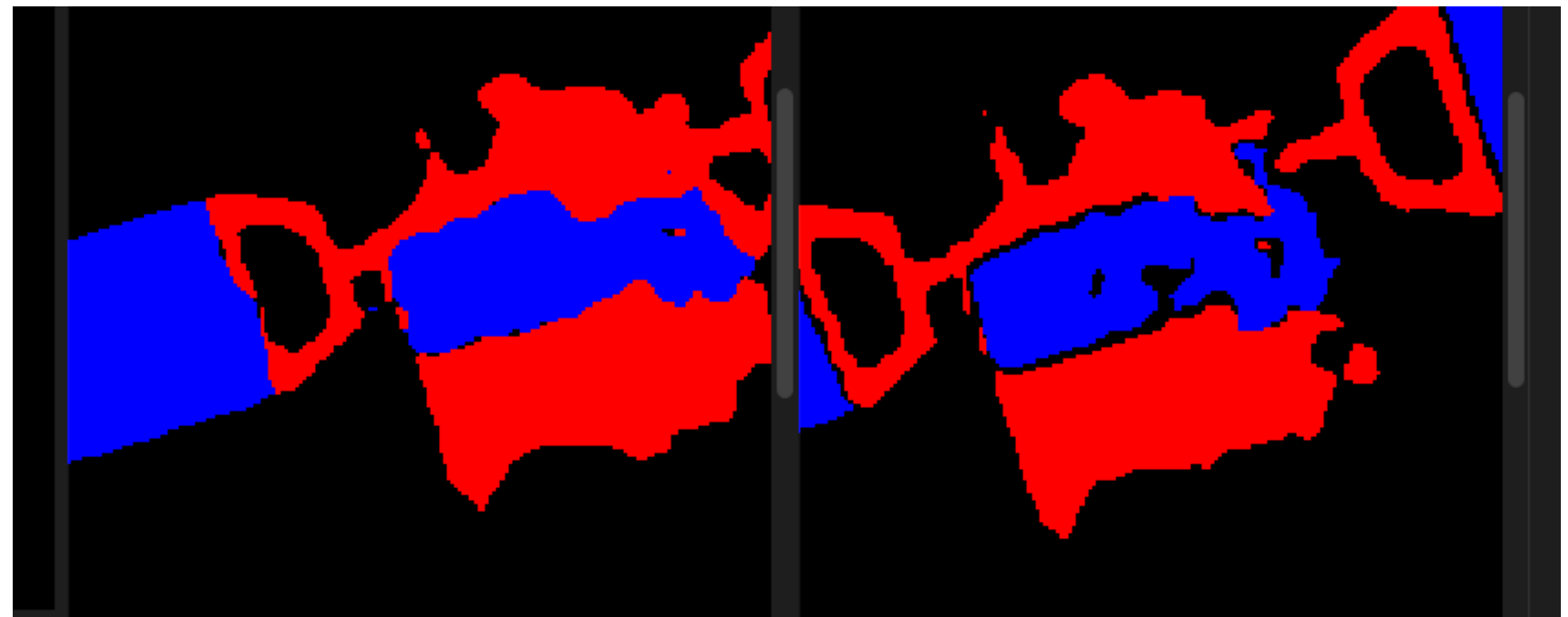
- Background dominates pixel distribution
- Standard cross-entropy treats all classes equally
- Leads to background-biased predictions

```
=== PER-IMAGE AVERAGE COMPOSITION (%) ===  
background: 92.99%  
body       : 5.78%  
panel      : 1.23%  
ignore     : 0.00% (avg fraction of ignore pi  
(segformer) 0 [earaya@iris-184 essentials](4655
```



Loss Function Modifications:

- Explored:
 - Weighted Cross-Entropy
 - Dice Loss
 - Combination of both
- Dice + WCE caused over-prediction of minority classes
- Final choice: Weighted Cross-Entropy only
- Class weights: (0.7, 1, 1)

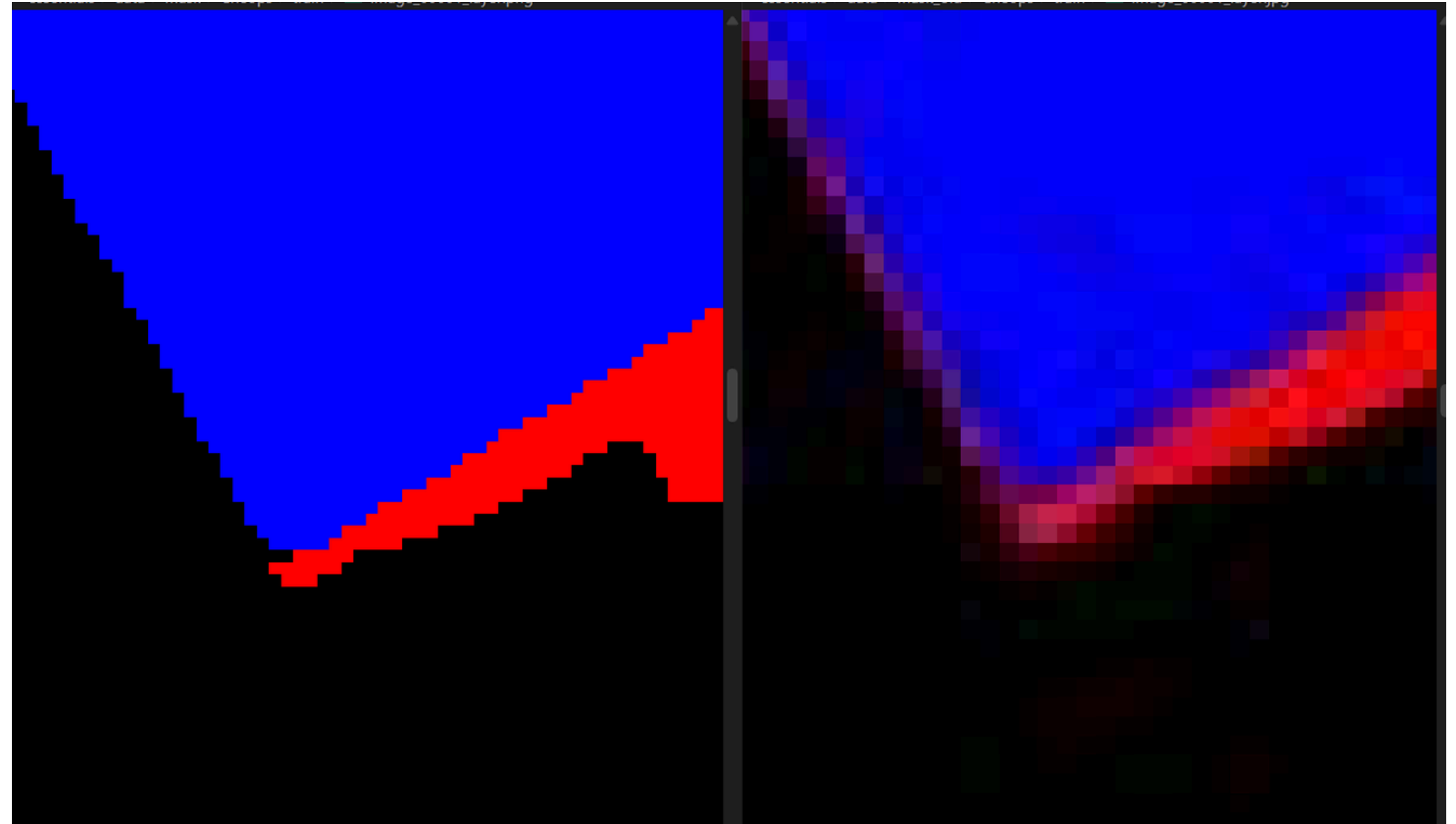


Mask Labeling Issue Discovery:

- Masks stored in JPG format
- Compression artifacts on boundaries
- Many pixels not purely red or blue
- Incorrectly labeled as background

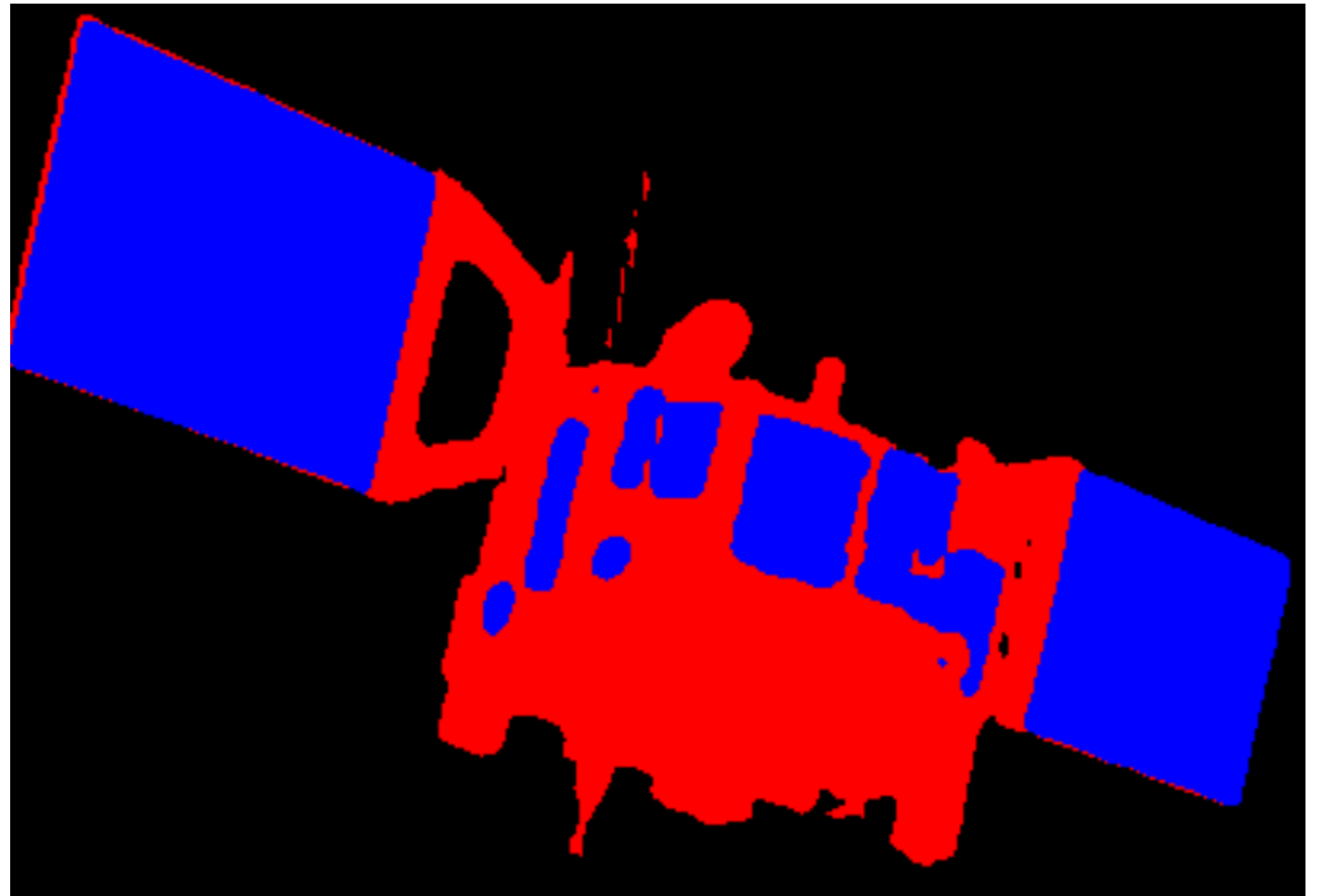
Input Mask Preprocessing Fix:

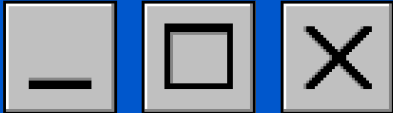
- Euclidean distance in RGB space
- Reference colors:
 - Body: (255, 0, 0)
 - Panels: (0, 0, 255)
 - Background: (0, 0, 0)
- Pixel assigned to the nearest class
- Masks saved as PNG



Performance After Preprocessing:

- Improved boundary accuracy
- Reduced background bias
- Codabench score increased to 95%
- Most significant performance gain came from preprocessing



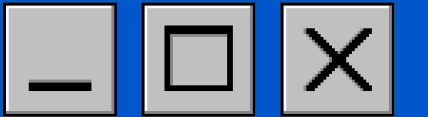


CONCLUSION

Model (setting)	Score
Detection	
Mask R-CNN (ensemble)	0.9452
DETR (ResNet-101)	0.6392
RT-DETR (ResNet-50-vd)	0.9674
Segmentation	
Mask R-CNN	0.64
DeepLabV3 (1024, 20 epochs)	0.91
SegFormer-B4 (baseline)	0.89
SegFormer-B4 (improved masks)	0.9496

Overall Results							
Task:				SPARK2024 Task			
#	Participant	Date	ID	Detection	Segmentation	Final Score	Detailed Results
	Alpha_25	2026-01-07 01:01	478271	0.9674	0.9496	0.9585	
	beta25_mics_25	2026-01-15 11:01	488870	0.9884	0.9185	0.9535	
	Epsilon_MICS_25	2026-01-16 01:05	489979	0.9769	0.9279	0.9524	





THANK YOU!

Shut Down...

